

Practical - 9

Practical Title: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem definition: Implement secure user registration by inserting user details into MySQL using MySQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

Theory:

1. Is password_hash() used before storing passwords?

Answer: Yes, password_hash() is used before storing the password. It converts the password into a secure hash, so the original password is not directly stored in the database.

2. Is duplicate email/username checked?

Answer: Yes, duplicate username and email are checked before inserting the new user. If the username or email already exists, the registration is stopped and an error message is shown.

3. Are frontend and backend validations consistent?

Answer: Yes, frontend and backend validations are consistent. Frontend validation checks things like required fields, valid email and minimum password length. Backend validation checks the same things again using PHP, making the registration more secure.

A. Implementation/ Output/Result (Keep only one option which is suitable to practical)

HTML tags used

Tag	Purpose
<html>	Defines the complete HTML document.
<head>	Contains information about the webpage, such as title and CSS.
<title>	Sets the title shown on the browser tab.
<style>	Contains CSS used to design the page.
<body>	Contains the visible content of the webpage.
<div>	Groups related elements together. Here it is used for the registration box.
<h2>	Displays a heading. Used for "StudentHub Registration".
<form>	Creates the registration form and sends the entered data to PHP.
<input>	Creates input fields for username, email and passwords.

Tag	Purpose
<button>	Creates the Register button.
<p>	Displays the registration message.
 	Adds a line break.

Important input attributes

Attribute	Purpose
type="text"	Creates a text field for username.
type="email"	Creates an email input field and checks email format on the frontend.
type="password"	Hides the password while entering it.
name	Gives a name to the input so PHP can access its value using \$_POST.
placeholder	Shows a hint inside the input field.
required	Makes the field compulsory.
maxlength	Limits the maximum number of characters.
minlength	Sets the minimum number of characters.
method="POST"	Sends form data using the POST method.
action=""	Submits the form to the same page.

PHP/SQL parts used

Function / Keyword	Purpose
new mysqli()	Connects PHP to the MySQL database.
\$_POST	Gets data submitted through the form.
trim()	Removes unnecessary spaces from username/email.
filter_var()	Validates the email format.
strlen()	Checks the length of the password.
prepare()	Creates a prepared SQL statement.
bind_param()	Safely passes values to the SQL query.
execute()	Executes the SQL statement.
store_result()	Stores the result of the duplicate-check query.
num_rows	Checks whether a matching username/email exists.
password_hash()	Converts the password into a secure hash before storing it.
PASSWORD_DEFAULT	Specifies the default secure hashing method.
htmlspecialchars()	Safely displays the message on the webpage.

A. Evidence of Work (Screenshots)

The first screenshot shows the 'StudentHub Registration' form with the following fields: 'Enter username', 'Enter email', 'Enter password', and 'Confirm password'. A 'Register' button is at the bottom.

The second screenshot shows the form after a successful registration. The fields are filled with 'diya123', 'diyatolia123@gmail.com', and two masked passwords. The 'Register' button is still present, and a message 'Username or email already exists.' is displayed below it.

The third screenshot shows the form with a validation error. The 'Enter email' field contains 'diya789'. A red error message box is displayed: 'Please include an '@' in the email address. 'diya789' is missing an '@'. The 'Register' button is still present, and a message 'Username or email already exists.' is displayed below it.

localhost/SecureRegister/

StudentHub Registration

diya789

diyatolia123@gmail.com

! Please lengthen this text to 6 characters or more (you are currently using 4 characters).

Register

Username or email already exists.

localhost/SecureRegister/

StudentHub Registration

Enter username

Enter email ! Please fill out this field.

Enter password

Confirm password

Register

Username or email already exists.

ecureregister/

StudentHub Registration

Passwords do not match.

A. GitHub Repository Link:

<https://github.com/diyatolia1212/StudentHub/tree/01fc189ba4fe3786f5a20921cc7ca936fb415d3e/SecureRegister>

B. Challenges Faced:

- Connecting the PHP program with the MySQL database.
- Checking whether the username or email already exists in the database.
- Making sure all required fields are filled correctly.
- Validating the email and password on both frontend and backend.
- Understanding how to use password_hash() to store passwords securely.

Learning Outcomes:

- Learned how to validate user input using frontend and backend validation.
- Learned how to check duplicate username and email before registration.
- Learned how to use password_hash() for secure password storage.
- Learned how to use prepared statements for database operations.
- Learned how to display suitable messages for successful and unsuccessful registration

Faculty Remarks: (Keep this blank for faculty)